

Home SOC Lab Runbook

Wazuh + Sysmon + Shuffle — Setup, Integration & Testing

Security Lab

2026-07-06

Table of Contents

Home SOC Lab Runbook.....	1
How to use this runbook	3
1. What you'll build.....	3
2. How detection → response works	4
3. Setup roadmap.....	5
4. Prerequisites checklist	6
Part A — Wazuh Manager (Ubuntu 24.04)	6
Part B — Windows 11 Agent	7
Part C — Sysmon (Windows 11).....	7
Part D — Confirm Sysmon flows into Wazuh.....	8
Part E — Community Sysmon rules on the manager (optional)	9
Part F — Shuffle (SOAR) on the same Ubuntu host	10
Part G — Wazuh ↔ Shuffle integration	11
Part H — Testing (end to end)	12
Part I — Monitoring Metasploitable2 (agentless, via syslog)	14
Part J — Consolidated troubleshooting	15

Home SOC Lab Runbook

Wazuh (SIEM/XDR) · Sysmon · Shuffle (SOAR) — end-to-end setup, integration & testing

Manager / SOAR host	Ubuntu 24.04 LTS (<MANAGER_HOSTNAME>, <MANAGER_IP>)
Endpoint	Windows 11 + Wazuh agent + Sysmon (<WINDOWS_HOSTNAME>, agent id 001)
Automation	Shuffle SOAR (Docker stack on the Ubuntu host)
Audience	Home-lab / learning SOC analyst

Version

1.0 · 2026-07-06

Sensitive values are redacted. IPs, API keys, webhook URLs, org IDs and hostnames appear as <PLACEHOLDERS>. Substitute your own values before use.

Placeholder key

Placeholder	Meaning
<MANAGER_IP>	IP of the Ubuntu Wazuh manager / Shuffle host
<MANAGER_HOSTNAME>	Ubuntu host's hostname
<WINDOWS_HOSTNAME>	Windows 11 endpoint hostname
<VIRUSTOTAL_API_KEY>	Your VirusTotal API key (manager only)
webhook_<SHUFFLE_WEBHOOK_ID>	UUID portion of your Shuffle webhook URL
<SHUFFLE_ORG_ID> / <ORBORUS_AUTH>	Shuffle organization ID / Orborus auth token
<METASPLOITABLE_IP>	Metasploitable2 host IP (isolated lab segment)
<LAB_SUBNET>	Isolated lab subnet (e.g. Metasploitable's network)
<FW_LAB_GW>	OPNsense gateway IP on the lab subnet

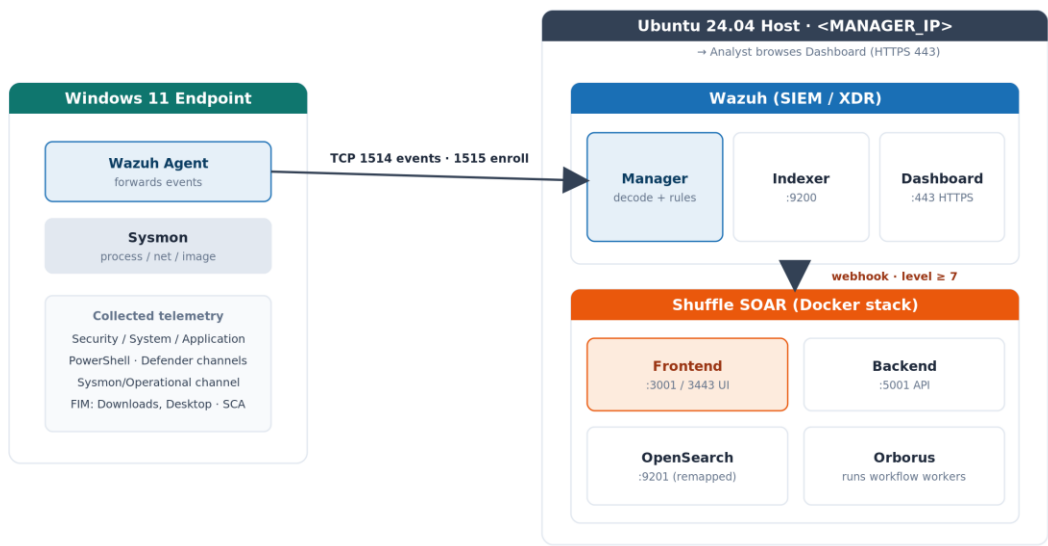
How to use this runbook

This is a build-and-verify guide. Work through **Parts A–H in order** — each part ends with a **Verify** checkpoint and, where relevant, the **Common issues** actually encountered during this build. Conventions:

- Commands prefixed for the Ubuntu manager run in a normal shell; Windows commands run in an **Administrator** PowerShell.
- Callouts: > **Note** (context), > **Warning** (do/don't), > **Verify** (confirm before moving on).
- Every issue we hit is captured in Part I (consolidated troubleshooting) and cross-referenced.

1. What you'll build

A Wazuh manager and a Shuffle SOAR instance on a single Ubuntu 24.04 host, monitoring a Windows 11 endpoint that runs the Wazuh agent plus Sysmon. **Wazuh detects; Shuffle automates the response.**



Lab architecture — endpoint telemetry flows to the Wazuh manager on the Ubuntu host, which forwards qualifying alerts to the co-located Shuffle SOAR stack.

Component roles

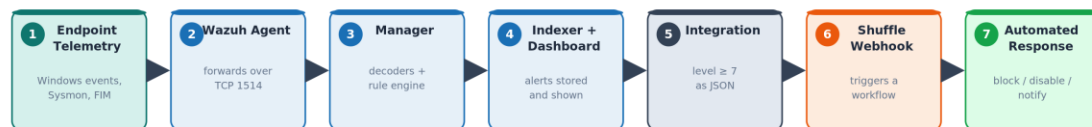
Component	Host	Purpose
Wazuh manager	Ubuntu 24.04	Rule/decoder engine, alerting, integrations (VirusTotal, Shuffle)
Wazuh indexer	Ubuntu 24.04	OpenSearch storage (127.0.0.1:9200)

Wazuh dashboard	Ubuntu 24.04	Web UI (https://<MANAGER_IP>)
Wazuh agent	Windows 11	Windows event channels, Sysmon, FIM, SCA
Sysmon	Windows 11	Deep endpoint telemetry (process, network, image load)
Shuffle SOAR	Ubuntu 24.04 (Docker)	Automation / response, UI on :3001

Ports

Port	Host	Purpose
1514/tcp	Manager	Agent event channel
1515/tcp	Manager	Agent enrollment (authd)
443/tcp	Manager	Wazuh dashboard (HTTPS)
9200/tcp	Manager	Wazuh indexer (loopback)
3001 / 3443	Manager	Shuffle UI (HTTP / HTTPS)
9201/tcp	Manager	Shuffle's OpenSearch (remapped from 9200 to avoid clash)

2. How detection → response works



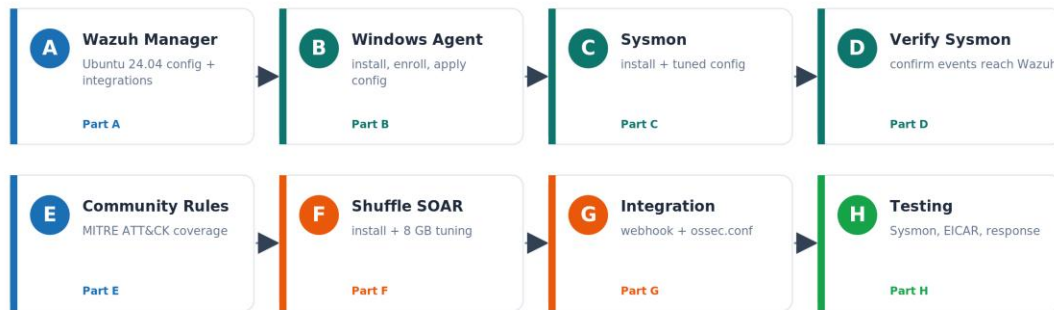
Detection-to-response pipeline: endpoint telemetry is collected by the agent, decoded and ruled on by the manager, stored and visualized, then forwarded to Shuffle where a workflow runs the response.

Endpoint events (Windows channels + Sysmon + FIM) are forwarded by the agent to the manager over TCP 1514. The manager decodes and evaluates them against its rules, stores alerts in the indexer, and shows them in the dashboard. Alerts at or above the configured level are POSTed as JSON to a Shuffle webhook, which triggers a workflow that performs the response.

3. Setup roadmap

Setup roadmap

Follow the parts in order. Each builds on the previous.



Setup roadmap — complete the eight parts in order; each builds on the previous.

4. Prerequisites checklist

- ☐ Wazuh central components (manager + indexer + dashboard) installed on Ubuntu 24.04.
 - ☐ Network reachability between Windows endpoint and manager on **1514/1515**.
 - ☐ A free **VirusTotal** API key (for malware detection).
 - ☐ Docker Engine (official packages) available on the Ubuntu host — see Part F1.
 - ☐ **RAM:** 12–16 GB recommended for Wazuh + Shuffle on one box. **8 GB works only with the tuning in Part F3.**
 - ☐ Administrator access on the Windows 11 endpoint.
-

Part A — Wazuh Manager (Ubuntu 24.04)

Goal: confirm the manager configuration used in this lab (SCA, FIM, rootcheck, vulnerability detection, VirusTotal + Shuffle integrations). A sanitized copy is in `ossec-manager.conf`.

A1. Key configuration blocks

Malware detection via VirusTotal — runs on the manager, applies to FIM events from the manager and every agent:

```
<integration>
  <name>virustotal</name>
  <api_key><VIRUSTOTAL_API_KEY></api_key>
  <group>syscheck</group>
  <alert_format>json</alert_format>
</integration>
```

Manager-side FIM realtime folder for local malware tests:

```
<directories realtime="yes" check_all="yes" report_changes="yes">/root/malware-lab</directories>
```

Create it: `sudo mkdir -p /root/malware-lab`.

Note: the VirusTotal key lives **only** on the manager. The Windows agent performs real-time FIM and forwards file hashes; the manager does the VT lookup. No VT key on the agent.

A2. Apply changes safely

```
sudo /var/ossec/bin/wazuh-analysisd -t      # validate config loads cleanly
sudo systemctl restart wazuh-manager
```

Warning: `wazuh-logtest` has **no** `-t` flag — it's interactive (reads log lines from stdin). Use `wazuh-analysisd -t` to validate configuration.

A3. Verify

```
systemctl is-active wazuh-indexer wazuh-manager wazuh-dashboard  
sudo ss -tlnp | grep -E ':443|:1514|:1515|:9200'
```

Verify: all three services active; ports 443/1514/1515/9200 listening.

Part B — Windows 11 Agent

Goal: install, enroll, and configure the agent so Windows events, Sysmon, FIM and SCA reach the manager. A sanitized copy of the config is in `ossec-agent.conf`. All commands run in an **Administrator** PowerShell.

B1. Install & enroll

```
# Download the agent MSI (match your manager's 4.x version):  
# https://packages.wazuh.com/4.x/windows/wazuh-agent.msi  
.\wazuh-agent.msi /q WAZUH_MANAGER="<MANAGER_IP>" WAZUH_AGENT_NAME="win11-hos  
t" WAZUH_REGISTRATION_SERVER="<MANAGER_IP>"
```

B2. Config highlights

- Manager connection to `<MANAGER_IP>:1514/tcp`.
- **FIM realtime** on user folders so malware tests are caught:

```
<directories realtime="yes" check_all="yes" report_changes="yes">C:\Users  
\*\Downloads</directories>  
<directories realtime="yes" check_all="yes" report_changes="yes">C:\Users  
\*\Desktop</directories>
```
- Registry autorun monitoring (Run / RunOnce / Services / Winlogon).
- Threat-hunting event channels: Security, System, Application, PowerShell/Operational, Windows Defender/Operational, and **Sysmon/Operational**.

B3. Start & verify

```
net start WazuhSvc # or: Restart-Service -Name wazuh
```

Verify: the agent shows **Active** in the dashboard within a minute. **Firewall:** the manager must allow inbound TCP **1514** and **1515** from the endpoint.

Part C — Sysmon (Windows 11)

Goal: deploy Sysmon with a curated config for deep endpoint telemetry.

C1. Pick ONE config (not both)

Sysmon uses a single active config; loading one replaces the other.

- **SwiftOnSecurity** (sysmonconfig-export.xml) — single, well-commented, great defaults. **Start here.**
- **Olaf Hartong sysmon-modular** — more granular, ATT&CK-mapped; more setup. Move to it later.

Save the chosen file next to Sysmon64.exe as sysmonconfig.xml.

C2. Install (must be elevated)

`.\Sysmon64.exe -accepteula -i sysmonconfig.xml`

Success prints Sysmon64 installed. and Sysmon64 started.

C3. Verify

`Get-Service Sysmon64`

`Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 5`

Verify: service **Running** and recent events returned. Update config later with
`.\Sysmon64.exe -c sysmonconfig.xml`.

C4. Common issue — install “fails” but says success

Symptom:

wevtutil.exe returned failure
 Event manifest installation failed with last error:
 The operation completed successfully.

followed by “Sysmon is not installed on this computer.” — a **failed install** (manifest didn’t register), despite the misleading last line.

- **Cause (almost always):** the prompt wasn’t elevated. Reopen PowerShell via **Run as administrator**, confirm with:

```
([Security.Principal.WindowsPrincipal][Security.Principal.WindowsIdentity]::GetCurrent()).IsInRole([Security.Principal.WindowsBuiltInRole]::Administrator)
```

Must print True, then re-run C2.

- **Still failing while elevated:** clean up and retry — `.\Sysmon64.exe -u` force then reinstall.

Part D — Confirm Sysmon flows into Wazuh

Goal: verify Sysmon events reach the dashboard.

Restart the agent so it picks up the now-populated channel, then generate a process-create event:


```
net stop WazuhSvc; net start WazuhSvc
whoami /all
powershell -enc VwByAGkAdABlAC0ASABvAHMAdAAgAGgAaQ== # harmless encoded "Write-Host hi"
```

Verify: Sysmon Event ID 1 (process create) alerts appear under **Threat Hunting**.

Part E — Community Sysmon rules on the manager (optional)

Goal: add MITRE ATT&CK detections on top of Wazuh's default Sysmon decoding. Run on the manager. A full community set is best added as its **own file** in /var/ossec/etc/rules/ (local_rules.xml is for your own handful of rules).

```
cd /var/ossec/etc/rules/
sudo curl -O https://raw.githubusercontent.com/socfortress/Wazuh-Rules/main/Sysmon/sysmon_rules.xml
sudo chown wazuh:wazuh /var/ossec/etc/rules/*.xml
sudo chmod 660 /var/ossec/etc/rules/*.xml
sudo /var/ossec/bin/wazuh-analysisd -t # validate BEFORE restart
sudo systemctl restart wazuh-manager
```

Warning: your own custom rules must use IDs **100000+**. Community rulesets occasionally collide with each other's IDs.

E1. Common issue — rules won't load (1202/1220)

```
wazuh-analysisd: ERROR: (1202): Configuration error at 'etc/rules/sysmon_rules.xml'.
wazuh-analysisd: CRITICAL: (1220): Error loading the rules: 'etc/rules/sysmon_rules.xml'.
```

Check the real reason: `sudo tail -n 30 /var/ossec/logs/ossec.log`. Usual causes:

- **Download isn't XML** — `curl -O` saved a 404/HTML page. `head -5 sysmon_rules.xml`; if you see `<!DOCTYPE html>`, re-download from the correct raw path.
- **Malformed XML** — `xmllint --noout sysmon_rules.xml` (file must be wrapped in `<group>...</group>`).
- **Duplicate/out-of-range IDs** — the log says Duplicate rule ID.

Recover: `sudo mv sysmon_rules.xml /tmp/ && sudo /var/ossec/bin/wazuh-analysisd -t`.

Part F — Shuffle (SOAR) on the same Ubuntu host

Goal: stand up Shuffle via Docker on the manager host, tuned to coexist with Wazuh.

Warning — sizing: on a single **8 GB** box, Wazuh + Shuffle will exhaust RAM unless you apply the tuning in **F3**. Untuned, the OOM killer takes down both and dashboards time out.

F1. Prerequisites (official Docker packages)

Do **not** mix Ubuntu's `docker.io` with Docker's `containerd.io` — `apt` throws `containerd.io Conflicts: containerd`.

```
apt list --installed 2>/dev/null | grep -E 'docker|containerd'
docker --version && docker compose version
```

- Compose plugin missing → `sudo apt install -y docker-compose-plugin`
- Docker not installed:

```
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin git
sudo systemctl enable --now docker
```
- Run docker without sudo: `sudo usermod -aG docker $USER && newgrp docker`

Kernel setting for OpenSearch (persisted):

```
echo 'vm.max_map_count=262144' | sudo tee /etc/sysctl.d/99-shuffle.conf
sudo sysctl -p /etc/sysctl.d/99-shuffle.conf
```

F2. Download & fix the port clash

```
git clone https://github.com/Shuffle/Shuffle && cd Shuffle
sudo chown -R 1000:1000 shuffle-database
# Remap Shuffle's OpenSearch off 9200 (Wazuh indexer owns it):
sed -i 's/- 9200:9200/- 9201:9200/' ~/Shuffle/docker-compose.yml
grep -n '920[01]' ~/Shuffle/docker-compose.yml      # verify 9201:9200
```

Warning: do **NOT** run `sudo swapoff -a` (the upstream guide suggests it). On this shared 8 GB box that removes the only buffer and triggers the OOM crash. Keep swap on — see F3.

F3. Single-box (8 GB) tuning — BEFORE first launch

Both OpenSearch instances default their heap to ~half of RAM and fight until the box thrashes. Apply all three:

1. Add a 4 GB swapfile (there's no swap by default):

```
sudo fallocate -l 4G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

2. Cap the Wazuh indexer heap — /etc/wazuh-indexer/jvm.options: -Xms1g / -Xmx1g, then sudo systemctl restart wazuh-indexer. **3. Cap Shuffle's OpenSearch heap** — under the opensearch service environment::

- "OPENSEARCH_JAVA_OPTS=-Xms512m -Xmx512m"

Budget after tuning: ≈ 5.8 GB baseline (OS/Docker 1.2, Wazuh indexer 1.5, manager 0.8, dashboard 0.7, Shuffle OpenSearch 1.0, Shuffle services 0.6), leaving ~2 GB for worker containers, with swap as backstop.

F4. Disable Swarm for workers (single-node)

Swarm-based worker deployment repeatedly breaks on one host (stale services, vxlan errors). Run workers as plain containers — under the orborus service environment: set:

- SHUFFLE_SWARM_CONFIG=

(default is run).

F5. Start & first-run

```
cd ~/Shuffle && docker compose up -d
docker compose ps          # all four services Up
free -h                   # available should stay > ~500Mi
```

Browse to **http://<MANAGER_IP>:3001** (HTTPS on 3443). Shuffle has **no default login** — the first account you create becomes admin.

Verify: UI loads and you can create the admin account. If the UI hangs on “waiting for the database”, OpenSearch isn't ready — check docker compose logs
opensearch --tail=40.

Part G — Wazuh ↔ Shuffle integration

Goal: forward qualifying alerts from Wazuh to a Shuffle webhook and confirm delivery.

G1. Create the webhook (Shuffle)

1. **Automate** → **Workflows** → + **Create Workflow** → name it → **Create from scratch**.
2. Drag a **Webhook** trigger onto the canvas; rename it Wazuh_alerts.
3. **Copy the Webhook URI** and click **Start** to arm it.
4. Connect it to a **Change Me** → **Repeat back to me** node; set its **Call** field to \$exec and **Save**. This echoes each alert so you can confirm delivery.

G2. Point Wazuh at Shuffle (manager)

Add to /var/ossec/etc/ossec.conf inside <ossec_config> (paste your URI with **no** trailing text after it):

```
<integration>
  <name>shuffle</name>
  <hook_url>http://<MANAGER_IP>:3001/api/v1/hooks/webhook_<SHUFFLE_WEBHOOK_ID>
</hook_url>
  <level>7</level>
  <alert_format>json</alert_format>
</integration>
```

- <level>7</level> forwards level 7+. Use <rule_id>92026</rule_id> for one rule, or temporarily <level>3</level> for a quick test.

```
sudo /var/ossec/bin/wazuh-analysisd -t
sudo systemctl restart wazuh-manager
```

G3. Verify delivery

```
sudo tail -f /var/ossec/logs/integrations.log
```

Verify: POSTs to the webhook URL appear; new runs show up in the Shuffle workflow (Source = webhook). Ensure the webhook is **Started** or alerts have nowhere to land.

Part H — Testing (end to end)

Goal: prove the full detect → forward → respond path.

H1. Agent reporting (do this first)

Dashboard → **Threat Hunting** → **Events**, agent 001, clear filters, last 24h, **Refresh**. >

Verify: any events = agent connected. Empty = fix connectivity (Active status, ports, firewall) > before continuing.

H2. Sysmon / threat hunting

Run the Part D commands; look for Sysmon **Event ID 1** alerts.

H3. Malware detection (FIM + VirusTotal) with EICAR

Warning: Windows Defender quarantines the EICAR test file within milliseconds — before FIM can hash it — so nothing reaches the Malware Detection module. Exclude the folder first.

```
Add-MpPreference -ExclusionPath "$env:USERPROFILE\Downloads"
Set-Content -Path "$env:USERPROFILE\Downloads\eicar.com" -Value 'X50!P%@AP[4\
PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*'
```

Verify: within a minute — an FIM “file added” alert, then a VirusTotal “malicious” alert. (Defender’s own detection is a windows_defender event under Threat Hunting, not the Malware Detection module, which filters rootcheck/virustotal/yara.)

H4. Shuffle response

Verify: the qualifying alert produced a new run in the Shuffle workflow, and the Change Me node output shows the alert JSON. Build the real response (block IP, disable account, notify) from there.

H5. SCA

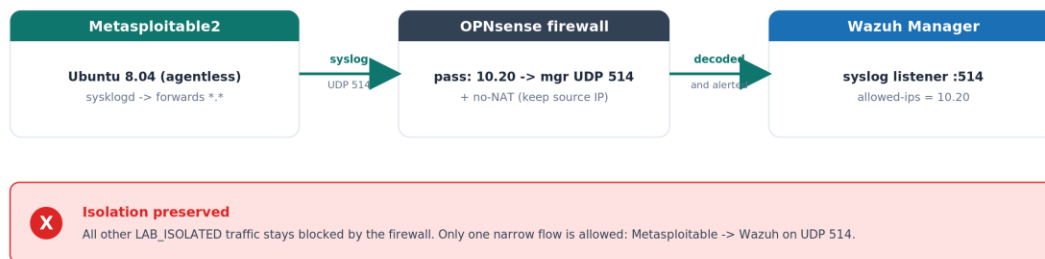
Dashboard → **Configuration Assessment** for the Windows CIS score.

Prepared by N.M.I Raisul Bari

Part I — Monitoring Metasploitable2 (agentless, via syslog)

Goal: monitor a legacy, deliberately-vulnerable host that **can't run the agent** (Ubuntu 8.04) and sits behind an **OPNsense firewall** on an isolated lab segment — by forwarding its syslog to the manager while keeping it otherwise isolated.

Metasploitable2 monitoring via syslog (through OPNsense)



Metasploitable2 forwards syslog through OPNsense to the Wazuh manager; a single UDP 514 flow is permitted while all other lab traffic stays blocked.

Why agentless: the Wazuh 4.x agent won't install on Ubuntu 8.04 (glibc/dependencies too old), and the box has no internet. It only needs to reach the manager on **UDP 514** over the LAN.

11. Manager — open a syslog listener

Add a **second** <remote> block (keep the existing secure/1514 one) to /var/ossec/etc/ossec.conf, right after it:

```
<remote>
  <connection>syslog</connection>
  <port>514</port>
  <protocol>udp</protocol>
  <allowed-ips><METASPLOITABLE_IP></allowed-ips>
</remote>
```

```
sudo ufw allow 514/udp
sudo /var/ossec/bin/wazuh-analysisd -t
sudo systemctl restart wazuh-manager
sudo ss -ulnp | grep :514          # wazuh-remoted must be listening on UDP 514
```

Warning: edit /var/ossec/**etc**/ossec.conf — **not** /var/ossec/ossec.conf (Wazuh ignores the latter). And put the **real IP** in <allowed-ips> with **no angle brackets** — a literal <METASPLOITABLE_IP> makes the XML invalid and wazuh-analysisd -t fails at “line 0”.

12. Metasploitable2 — forward its logs

```
ps aux | grep -E 'syslogd|rsyslog' | grep -v grep          # Ubuntu 8.04 = syslogd (/sbin/syslogd)
```

```
echo -e '*.*\t@<MANAGER_IP>' | sudo tee -a /etc/syslog.conf # real TAB before '@'
sudo /etc/init.d/syslogd restart
logger "wazuh syslog test"
```

Make the default gateway **persistent** — add gateway <FW_LAB_GW> under the static iface stanza in /etc/network/interfaces (a runtime route add is lost on reboot).

I3. OPNsense — allow only syslog, keep isolation

1. **Firewall rule** on the lab interface, placed **above** the LAB_ISOLATED → * block rule (first-match order matters): Pass · UDP · Source <METASPLOITABLE_IP> · Destination <MANAGER_IP> · Dest port 514.
2. **NAT exemption** so the manager sees the real source IP. Without it, packets arrive from the firewall's WAN address and <allowed-ips> won't match (the manager silently drops them): **Firewall** → **NAT** → **Outbound** → **Hybrid mode**, add a manual rule — Interface **WAN**, Source <LAB_SUBNET>, Destination <MANAGER_IP>, **Do not NAT** ☒.

I4. Verify

manager — source MUST be <METASPLOITABLE_IP>, not the firewall's IP
 sudo tcpdump -ni any udp port 514

(optional) see ALL received logs, not just rule matches
 sudo sed -i 's#<logall_json>no</logall_json>#<logall_json>yes</logall_json>#'
 /var/ossec/etc/ossec.conf
 sudo systemctl restart wazuh-manager
 sudo tail -f /var/ossec/logs/archives/archives.json | grep <METASPLOITABLE_IP>
 >

Generate an unmistakable alert — a few failed SSH logins — then check **Threat Hunting → Events**:

```
ssh baduser@<METASPLOITABLE_IP> # wrong password x3-4
```

Verify: tcpdump source is <METASPLOITABLE_IP>; the SSH failures appear in the dashboard attributed to that IP. logall_json grows fast — turn it back off once satisfied.

Security: Metasploitable2 is deliberately vulnerable — keep it on the isolated segment. The only egress you opened is syslog to the manager.

Part J — Consolidated troubleshooting

Every issue encountered during this build, with its fix.

Symptom	Root cause	Fix
---------	------------	-----

wevtutil returned failure / "Sysmon is not installed"

Install not elevated

Run PowerShe

Prepared by N.M.I Raisul Bari